WOOBI — MONOREPO ARCHITECTURE REPORT

Enterprise Modularization, Multi-Process Safety, Prevention Mechanisms & Operations Guide

ORGANIZATION	CANONICAL REPO	RELEASE BRANCH	COMMIT SHA
Troyz Solutions	Troyz-Solutions/Woobi	feature/phase-9-production-readiness	7035ad1 (Phases 1–9)

642 / 642	14 / 14	0
AUTOMATED TESTS PASSED (31 SUITES)	REAL HTTP E2E SCENARIOS PASSED	MODULE BOUNDARY VIOLATIONS

1. Executive Overview: What Was Accomplished

Woobi has completed a full 10-phase enterprise monorepo transformation. The codebase was re-architected from legacy unsegmented monolithic scripts into a clean, modular corporate monorepo structured across **5 physical domains**. All historical features (3D studio, wardrobe, lighting, mirrors, avatar rigs, recommendation algorithms, lookbook storage, and mobile Android WebView packaging) were fully preserved and verified with 0 regressions.

Key Milestone Deliverables:

- **5 Physical Architectural Domains** established with strict public interfaces (`index.js`).
- **Master Test Runner** upgraded to 31 test suites covering 642 unit, integration, and user-journey tests.
- **Real Backend HTTP Suite** covering 14 live network scenarios over local socket connections.
- **Architectural Boundary Enforcer** (`validate-boundaries.ps1`) blocking deep private imports, circular dependencies, and browser-to-backend leakage.
- **Multi-Process Database Durability** implementing kernel-level OS file locking and automatic `.bak` recovery.
- **Upstream Integration PR #6 & PR #7** published on GitHub with unbroken linear ancestry.

2. Monorepo Structural Anatomy

The repository enforces strict separation of concerns across 5 domains:

```
Woobi/
├── modules/
│   ├── core/                       # Domain 1: Authoritative Reactive Store & Routing
│   │   ├── state/Store.js          # Central reactive store & localStorage persistence
│   │   ├── navigation/Router.js    # Screen routing, lifecycle hooks & DOM bridges
│   │   └── index.js                # Public Core interface
│   ├── 3d-studio/                  # Domain 2: Physical Three.js 3D Studio Engine
│   │   ├── stage/                  # Podium, ground plane, shadows, backdrops
│   │   ├── wardrobe/               # Wardrobe closet 3D asset & door kinematics
│   │   ├── mirror-vanity/          # Vanity table, mirror reflection & vanity lighting
│   │   ├── avatar/                 # Mannequin 3D mesh, bone rigs, skin shaders
│   │   ├── lighting/               # 3-point studio lighting (Key, Fill, Rim)
│   │   ├── theme/                  # Environmental backdrop materials & studio themes
│   │   ├── camera/                 # Cinematic camera presets (Overview, Closeup, Focus)
│   │   └── index.js                # Unified Studio3D orchestrator (182 Tests)
│   ├── 3d-studio-features/         # Domain 3: High-Level User 3D Studio Features
```

```
│   │   ├── virtual-try-on/          # Layered garment fitting, equips & catalog
│   │   ├── garment-interactions/    # Three.js raycasting, pointer selection & hover highlights
│   │   ├── outfit-recommendations/  # Color harmony pairing & occasion outfit rules
│   │   ├── avatar-customization/    # Skin tones, hairstyles, gender presentation
│   │   ├── animations/              # Idle breathing, fashion poses & transitions
│   │   ├── saved-looks/             # Lookbook manager, outfit serialization & recall
│   │   └── index.js                 # Unified Features orchestrator (159 Tests)
│   ├── settings/                    # Domain 4: User Configuration & Preferences
│   │   ├── account/                 # Profile metadata, email updates & auth bridges
│   │   ├── appearance/              # Theme selection & lighting preset persistence
│   │   ├── privacy/                 # Consent toggles, cache purging & JSON data export
│   │   ├── notifications/           # Streak alerts, style briefings & reminder schedules
│   │   ├── performance/             # Graphics presets, shadow toggles & battery saver
│   │   └── index.js                 # Unified Settings orchestrator (134 Tests)
│   └── backend/                     # Domain 5: Backend, Auth, DB & Security Architecture
│       ├── security/                # Rate limiting, security headers, CORS, sanitization
│       ├── auth/                    # Password hashing, JWT tokens, persistent revocation
│       ├── database/                # OS file locks, atomic rename writes, .bak auto-healing
│       ├── storage/                 # Upload sandboxing, image mime checks & asset storage
│       ├── api/                     # REST router, parameter parser, ApiClient SDK
│       └── index.js                 # Unified Backend service & SDK (140 Tests)
├── scripts/
│   ├── testing/                     # Master Runner, HTTP E2E, User Journeys & Concurrency
│   └── validation/                  # AST-based architectural boundary validator
├── android-app/                     # Native Android Client (Gradle / Kotlin / WebView)
├── docs/PRODUCTION_RUNBOOK.md       # Nginx TLS proxy, PM2, disaster backups, rollback
├── CONTRIBUTING.md                  # Contributor handbook, branch rules & code tutorial
└── ecosystem.config.cjs             # PM2 process manager configuration (instances: 1)
```

3. How the Architecture Works: Interactions & Lifecycle

1. Core State & Routing

`Store.js` provides a reactive single source of truth. All user actions (changing outfits, selecting occasions, recording daily streaks) update state and trigger subscribers. State automatically writes to `localStorage` with legacy key migration adapters.

2. 3D Studio Engine

The Three.js engine binds lighting, camera framing, stage geometry, and rigged avatar meshes. When garments are equipped, geometry is dynamically bound to the avatar skeleton and shaders automatically adapt to the active studio lighting preset.

3. High-Level Studio Features

Virtual Try-On disposes of replaced garment meshes cleanly from GPU memory to prevent WebGL memory leaks. Garment interactions run raycasting on pointer move for hover states without stalling camera orbit controls.

4. Browser-Safe Backend Client SDK

Frontend code never accesses server internals directly. Communication occurs exclusively through `createApiClient()` over HTTP REST. Responses use standard JSON envelopes (`{ success, data, error }`).

4. How It Prevents Issues: Safety, Durability & Hardening

HAZARD / RISK	PREVENTION MECHANISM	ENGINEERING VERIFICATION
Circular Dependencies & Module Leaks	AST Boundary Enforcer (`validate-boundaries.ps1`) enforces public-only `index.js` imports and blocks browser imports of backend files.	Automated CI step checks all 5 primary domains. 0 VIOLATIONS
Multi-Process Concurrent Write Collisions	Kernel-level OS directory locking (`.oslock` via `fs.mkdirSync`) + temporary staging files with atomic `fs.renameSync` .	`test-multiprocess-concurrency.js` executes 4 simultaneous worker processes with 0 lost writes.
Sudden Server Crash / Corrupted JSON	Dual snapshot rotation writes `.bak` copies before any file modification. If JSON parsing fails, `readCollection` auto-restores from backup.	Simulated crash test corrupts primary JSON; auto-healing recovers data and emits `RECOVERED_FROM_BACKUP` .
Stale Multi-Process Cache Drift	`readCollection()` verifies physical file `mtimeMs` against in-memory cache timestamps. Disk changes immediately invalidate cache.	Cross-process revocation test confirms changes by Worker 1 are immediately observed by Worker 2.

HAZARD / RISK	PREVENTION MECHANISM	ENGINEERING VERIFICATION
Unauthorized Entitlement Tampering	`NODE_ENV === 'production'` strictly blocks developer verification bypass keys (`DEVELOPMENT_BYPASS_KEY`) for premium features.	HTTP scenario 14 confirms dev bypass keys return HTTP 403 Forbidden in production mode.
Mobile WebGL Context Dropping	Three.js lifecycle handlers listen for `webglcontextlost` to stop render loops, and `webglcontextrestored` to rebuild scene geometry.	User Journey 7 verifies context restoration without state loss or application crashes.

5. How to Use & Operate Woobi: Developer & Operator Guide

5.1 Running Locally & Verification Commands

```
# 1. Install Dependencies
npm install

# 2. Run the Full 31-Suite Regression Suite (642 Tests)
npm test

# 3. Run Real Backend HTTP Integration Scenarios (14 Scenarios)
npm run test:e2e

# 4. Validate Monorepo Boundaries & Circular Dependencies
npm run validate:boundaries

# 5. Run Multi-Process Concurrency & OS Lock Tests
node --test scripts/testing/test-multiprocess-concurrency.js

# 6. Start Local Development Server (Port 8080)
npm start
```

5.2 Building the Native Android Client

```
cd android-app
./gradlew assembleDebug

# Output APK:
# android-app/app/build/outputs/apk/debug/app-debug.apk
```

5.3 Production Deployment (PM2 Single-Writer Mode)

Because Woobi uses an atomic local-first JSON database, the application is strictly configured for **single-process execution** (`instances: 1` , `exec_mode: 'fork'`) in PM2. Multi-worker cluster mode is prohibited to eliminate concurrent worker file contention.

```
# Start application under PM2 in production single-writer mode
pm2 start ecosystem.config.cjs --env production

# Persist PM2 across host server reboots
pm2 save
pm2 startup

# Inspect live cluster logs and health
pm2 logs woobi-backend
pm2 status
```

6. Canonical GitHub Integration & Release Status

> **Production Gate Status: GATE HOLD (Production Deployment Not Authorized)**
> The codebase is fully ready for **Staging** deployment. However, production deployment remains held pending human CODEOWNERS review of PR #6 and real-device Android qualification.

PULL REQUEST	TARGET BASE	HEAD BRANCH	SCOPE / DELIVERABLES	STATUS
PR #6 Troyz-Solutions/Woobi#6	`main`	`integration/3d-studio-modular-monorepo`	Phase 1–4 Foundation (Stage, Wardrobe, Vanity, Avatar, Lighting, Theme, Camera).	AWAITING HUMAN CODEOWNER APPROVAL
PR #7 Troyz-Solutions/Woobi#7	`integration/3d-studio-modular-monorepo`	`feature/phase-9-production-readiness`	Phases 5–9 Complete Architecture, User Journeys, Multi-Process Durability, Contributor Guide & Runbooks.	OPEN (STACKED ON PR #6)

Release Sequence Roadmap

1. Designated code owners (`mosesacknals139` , `04suriya07-spec`) review and merge **PR #6** into `main` .
2. On GitHub, update **PR #7**'s base branch from `integration/3d-studio-modular-monorepo` to `main` .
3. Review and merge **PR #7** into `main` .
4. Deploy `main` commit to Staging server with PM2 single-writer configuration.
5. Sideload `app-debug.apk` onto physical Android devices to verify Three.js 60 FPS performance and touch controls.
6. Generate production keystore (`keystore.properties`), assemble signed release AAB, and promote to production.